

Identificadores Estáveis em Três Níveis

Camada de Indireção com UUIDs Descentralizados e Mapeamento de Densidade Numérica

Projeto: Aresta Climb • Repositório: aresta_db • Autor: Equipe de Arquitetura Aresta • Data: Outubro de 2026 • Versão: 1.0

1. Resumo Executivo: Por que esta arquitetura é necessária?

Esta especificação define a transição do modelo histórico de referência textual de dados para uma **arquitetura de identificadores estáveis em três níveis**. O objetivo primário é conciliar três pilares fundamentais da plataforma:

- O Mundo Físico da Montanha:** Placas permanentes de aço inoxidável gravadas a laser parafusadas na rocha, contendo QR codes ultra-curtos e robustos para leitura rápida sob sol intenso (ex: `app.arestaclimb.com/12/35`).
- A Engenharia Offline-First:** Sincronização rápida em conexões lentas de montanha via PowerSync e Supabase, exigindo chaves primárias e relacionamentos com inteiros de 4 bytes (`int32`) no SQLite, eliminando o overhead de strings.
- A Colaboração Descentralizada no Git:** Múltiplos conquistadores e contribuidores criando vias e croquis em paralelo em branches concorrentes, sem risco de conflito destrutivo de merge nem colisão de IDs numéricos.

2. As 7 Armadilhas (Pitfalls) das Abordagens Ingênuas

A complexidade de introduzir tabelas de mapeamento e UUIDs descentralizados justifica-se diretamente pelas armadilhas graves das abordagens simplórias:

Pitfall 1: O Modelo Histórico de Referências por Nomes de Texto

A abordagem ingênua: Referenciar vias em mapas por strings textuais compostas: (`setor: "Gruta", escalada: "Sombra Fresca"`).

Por que falha: Se o conquistador corrige um erro de digitação na via, **todas as referências em múltiplos arquivos de mapas quebram**. Caçar referências em arquivos Markdown no editor desktop gera complexidade acidental (o comando `CmdRenomearEscalada` precisava varrer todo o disco). Além disso, a resolução no app mobile exige comparações de strings caras em tempo de execução e é inviável gravar caminhos textuais longos em placas físicas na rocha.

Pitfall 2: IDs Numéricos Incrementais Diretos no Conteúdo (Sem Indireção)

A abordagem ingênua: Fazer o editor ou usuário preencher `id: 15`, `id: 16` diretamente dentro de `setor.md` ou `croqui.yaml`.

Por que falha: Colisão inevitável de IDs em branches paralelos. Se o Colaborador A criar a via "Aresta Sul" recebendo o próximo ID livre (16) e o Colaborador B criar a via "Fenda Central" em seu fork também recebendo o ID 16, ao abrirem Pull Requests haverá **colisão silenciosa**. Mais grave: se os mapas já desenharem traçados apontando para `alvo_id: 16`, renumerar uma das vias exigiria caçar e reescrever o ID em dezenas de arquivos de mapas e pontos de interesse.

Pitfall 3: Usar Apenas UUIDs/NanoIDs em Todo Lugar (Sem IDs Numéricos)

A abordagem ingênua: Adotar UUIDs em todo o ecossistema e nunca gerar números inteiros.

Por que falha: Inviabiliza a sinalização física e degrada a performance do app. Uma URL como `app.arestaclimb.com/c78a-9b12/x89p-1234` é longa, gerando um QR code de alta densidade que não pode ser gravado com nitidez no aço inox nem escaneado com facilidade com lentes arranhadas ou luz solar direta. No banco SQLite/PowerSync móvel, joins entre inteiros de 4 bytes são ordens de magnitude mais rápidos e ocupam uma fração do espaço de índices sobre strings UUID.

Pitfall 4: Espaço Numérico Único para Vias e Pontos de Interesse (POIs) de Mapas

A abordagem ingênua: Compartilhar o mesmo contador numérico entre escaladas e nós de linhas vetoriais do mapa.

Por que falha: Em croquis com traçados vetoriais detalhados, um único mapa de setor pode conter 500 nós intermediários de linhas e setas. Se os POIs consumissem o mesmo espaço de IDs, a via número 15 pularia para o ID 516 simplesmente porque o editor traçou linhas antes de cadastrar a próxima via! Isso geraria placas de aço na montanha com números gigantescos e não-mnemônicos em setores pequenos.

Pitfall 5: Exigir Sequencialidade Estrita (Contiguidade sem Gaps) e Contadores Globais

A abordagem ingênua: Forçar que os IDs sejam estritamente `1, 2, 3... N` sem furos e manter campos como `proximo_croqui_id` no catálogo.

Por que falha: Se uma via for desativada ou conquistada por engano e excluída, forçar contiguidade exigiria **renumerar todas as vias posteriores**. Isso **invalidaria permanentemente placas físicas de aço parafusadas na rocha** com QR codes gravados a laser! Além disso, contadores de sequência em arquivos centrais causam conflitos permanentes no Git. IDs devem ser *estáveis e imutáveis*, e furos/gaps decorrentes de exclusões são 100% naturais e desejáveis.

Pitfall 6: Acoplamento do Serving do App com Migrações Internas do Database

A abordagem ingênua: Fazer qualquer script em `migracoes/` incrementar a versão pública do catálogo (`kDataVersion = 4 → 5`).

Por que falha: O aplicativo móvel monitora `kDataVersion`. Se uma migração interna no formato Markdown da pasta `database/` incrementar essa versão, **milhares de usuários com o app em falésias remotas teriam seus croquis salvos invalidados**, bloqueados por uma tela de migração exigindo internet para baixar todo o catálogo novamente. A camada de serving de produção deve permanecer em `v4` enquanto o formato compilado `.binarypb` for retrocompatível.

Pitfall 7: Conflitos de Merge Destrutivos em Arquivos YAML (Git Conflict Markers)

A abordagem ingênua: Resolver merges de tabelas de IDs na mão ou depender de ferramentas externas de Git.

Por que falha: Conflitos de merge em arquivos de mapeamento geram marcadores como `<<<<<< HEAD` que quebram parsers YAML normais. Uma resolução incorreta na mão pode sobrescrever acidentalmente o ID de uma via já publicada na base oficial. O sistema exige uma biblioteca autônoma com resolução semântica determinística.

3. A Solução: Arquitetura em Três Níveis de Indireção

A arquitetura resolve simultaneamente todos os pitfalls através da separação explícita entre a **identidade lógica descentralizada** (UIDs) e a **representação física e compacta** (IDs numéricos):

[ARQUIVOS FONTE NO DATABASE/]
Setores, Vias e POIs usam UUIDs

[TABELAS DE MAPEAMENTO YAML]
Apenas pares (id, uid) em 3 níveis

```
setor_imigrantes.md
uid: "k7X9-m2P1-qRt4" (NanoID 12c)
mapas:
  referencias:
    - alvo_uid: "a1B2-c3D4-e5F6"
  pontos_uids: ["x9Y1-w2Z3-v4U5"]
  pontos_de_interesse:
    - uid: "x9Y1-w2Z3-v4U5"
    rotulo: "11"
```

```
database/ids_globais.yaml
- id: 12 <-- Nível 1: croqui_id_num
  uid: "c8B1-m4P2-zQ99"

database/<croqui>/ids_entidades.yaml
- id: 35 <-- Nível 2: entidade_id_num
  uid: "a1B2-c3D4-e5F6" (Via)

database/<croqui>/ids_pontos.yaml
- id: 104 <-- Nível 3: ponto_id_num
  uid: "x9Y1-w2Z3-v4U5" (POI/Nó)
```

COMPILADOR (deploy_generated.py)

- Omite todos os UUIDs (economia bytes)
- Injeta IDs numéricos nos 3 níveis
- Popula strings legadas em memória

```
compilado.binarypb (App Móvel)
- croqui_id_numerico: 12
- alvo_id: 35
- pontos_ids: [104]
- label: "11" / rotulo: "11"
- escalada: "Sombra Fresca" (retrocompat)
```

Placa de Aço Inox Gravada a Laser
"app.arestaclimb.com/12/35"
QR Code ultra-curto, limpo e legível
Mesmo em condições severas de montanha

4. Detalhamento dos Três Níveis de Identificadores Numéricos

Nível	Campo Protobuf	Tabela YAML	Escopo	Propósito e Comportamento
Nível 1 (Global)	croqui_id_numerico	database/ids_globais.yaml	Global	Identificador único global do croqui/pico. Utilizado na rota base das URLs de acesso e placas físicas (ex: /12/...). Não requer contadores centrais em Protobuf.
Nível 2 (Local - Entidades)	entidade_id_numerico (Escalada.id, Setor.id, Grupo.id, alvo_id)	database/<croqui>/ids_entidades.yaml	Local Croqui	Identificador de vias, setores e grupos. Mantém números compactos para as placas físicas de aço na rocha (ex: via 35 → app.arestaclimb.com/12/35). Gaps permitidos.
Nível 3 (Local - Pontos)	ponto_id_numerico (PontoDeInteresse.id, pontos_ids)	database/<croqui>/ids_pontos.yaml	Local Croqui	Identificador exclusivo de nós de traçados e POIs visuais de mapas. Impede que centenas de nós vetoriais inflem os IDs de vias ou alterem a numeração de placas.

5. Resolução Semântica de Conflitos no Git (Técnica do Split de Visões)

Quando múltiplos colaboradores criam vias ou croquis em paralelo, o Git insere marcadores de conflito em arquivos YAML:

```
ids_entidades:
<<<<<< HEAD
- id: 35
  uid: "a1B2-c3D4-e5F6"
=====
- id: 35
  uid: "w8K9-z1X2-y3T4"
>>>>>> branch_do_usuario
```

A biblioteca autônoma `scripts/gerenciar_ids_numericos_lib.py` resolve isso de forma determinística sem tocar no Git CLI:

- Split de Visões em Memória:** O arquivo textual é desmembrado em dois documentos YAML válidos: a *Visão A (HEAD)* e a *Visão B (Conflitante)*.
- Preservação da Base Oficial:** Todos os IDs da Visão A (HEAD) são aceitos como verdade oficial imutável.
- Alocação para Entradas Novas:** O algoritmo detecta que `w8K9-z1X2-y3T4` é um UID novato da Visão B. Ele anexa essa entrada ao final da lista atribuindo o próximo inteiro livre disponível (ex: `id: 36`).
- Gravação Limpa:** O arquivo é regravado sem marcadores de conflito. **Nenhum arquivo Markdown de vias ou mapas precisa ser alterado**, pois todo o conteúdo continua referenciando o UID imutável!

6. Governança de Migrações: Protegendo o Serving de Produção

Mecanismo Database-Only (AFETA_VERSAO_SERVING = False)

Para que croquis experimentais e arquivos de desenvolvimento sejam migrados para a nova estrutura sem forçar atualizações no app mobile, introduzimos a flag declarativa:

```
# Em migracoes/0005_migrar_uids_e_rotulos.py:
AFETA_VERSAO_SERVING: bool = False
```

O módulo `serving/update_serving.py` ignora migrações marcadas com essa flag no cálculo de `get_db_version()` . A versão pública do serving permanece rigorosamente em `v4` , enquanto o motor `scripts/migrador.py` executa a migração nos dados e atualiza `ultima_migracao: 5` em `croqui.yaml` .

7. Testes de Contrato Arquiteturais Inegociáveis

Para garantir que o código legado nunca retorne ao repositório, duas suítes de testes de contrato rodam obrigatoriamente no CI:

- 1. **Contrato da Pasta** `database/` (`tests/contrato_database_uids_test.py`): Varre todos os `*.md` e `croqui.yaml`. Falha imediatamente se encontrar o campo `id:` direto nas entidades (permitido apenas em `ids_*.yaml`), referências contendo `escalada:`, `setor:` ou `grupo:`, ou pontos de interesse contendo `label:` (exigido `rotulo:`).
- 2. **Contrato do Editor Desktop** (`tests/contrato_editor_serializacao_test.py`): Garante que as rotinas de serialização de novos croquis e modificações nunca emitem campos depreciados.

8. Resumo Comparativo: Abordagens Ingênuas vs. Nossa Arquitetura

Dimensão de Avaliação	Abordagem Ingênua (Nomes ou IDs Diretos)	Nossa Arquitetura (UIDs + 3 Níveis)
Renomeação de Vias	Quebra mapas ou exige varredura em massa no disco	Edição local de 1 linha (mapas apontam para UIDs imutáveis)
Conflitos de Merge no Git	Colisão destrutiva de IDs; renumeração em cascata	Zero conflito em conteúdo ; merge automático nas tabelas YAML
Placas Físicas na Rocha	URLs longas (nomes) ou números inflados (POIs misturados)	URLs ultra-curtas e compactas (<code>app.arestaclimb.com/12/35</code>)
Performance Móvel & Banda	Comparações de strings lentas ou UUIDs pesados no binário	O(1) com inteiros de 4 bytes ; UIDs omitidos do <code>.binarypb</code>
Impacto no Usuário Móvel	Cada refatoração interna força redownload total no app	Zero impacto no app (serving permanece estável em <code>v4</code>)

9. FAQ Rápido para Desenvolvedores

- **P: Devo colocar `id` quando cadastrar uma via ou setor no Markdown?**
R: Não! No Markdown/YAML de conteúdo, use apenas `uid: "xxxx-xxxx-xxxx"`. O ID numérico é atribuído pelo compilador/tabela.
- **P: Por que não usamos apenas UUID v4?**
R: O NanoID Base62 com 12 caracteres (`xxxx-xxxx-xxxx`) é legível por humanos, sem caracteres especiais, e possui mais de 3×10^{21} combinações (colisão nula).
- **P: Se uma via for deletada, o ID dela é reutilizado?**
R: Não. Furos/gaps são permanentes para garantir que placas físicas já gravadas e instaladas na rocha continuem válidas e imutáveis.
- **P: Como os clientes legados continuam funcionando se deletamos `escalada` dos mapas?**
R: O compilador `deploy_generated.py` preenche os campos legados em memória durante a compilação de `compilado.binarypb`.